

```

import os
import json
import logging
from datetime import datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, Conte

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ——— Данные —————

PEOPLE = {
    "ilyas": {"name": "Ильяс", "emoji": "👤"},
    "azhar": {"name": "Ажар", "emoji": "👤"},
    "karina": {"name": "Карина", "emoji": "👤"},
}

TASKS = {
    "kitchen": {"person": "ilyas", "name": "Помыл кухню", "emoji": "🧼"},
    "dishes": {"person": "ilyas", "name": "Помыл посуду", "emoji": "🍽️"},
    "trash": {"person": "azhar", "name": "Вынес мусор", "emoji": "🗑️"},
    "shoes": {"person": "azhar", "name": "Убрал обувь", "emoji": "👟"},
    "toys": {"person": "karina", "name": "Убрала игрушки", "emoji": "🧸"},
    "general": {"person": "karina", "name": "Генеральная уборка", "emoji": "🧹"},
}

DATA_FILE = "scores.json"

# ——— Хранилище —————

def load_data():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return {"scores": {"ilyas": 0, "azhar": 0, "karina": 0}, "history": [], "today": 0}

def save_data(data):
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def get_loser(scores):
    return min(scores, key=scores.get)

```

— Команды —

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    keyboard = [
        [InlineKeyboardButton("📅 Задачи", callback_data="show_tasks"),
         InlineKeyboardButton("🏆 Счёт", callback_data="show_scores")],
        [InlineKeyboardButton("📖 История", callback_data="show_history")],
    ]
    await update.message.reply_text(
        "🏠 *Домашний трекер уборки*\n\nВыбери действие:",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup(keyboard)
    )
```

```
async def scores_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await send_scores(update.message.reply_text)
```

```
async def tasks_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await send_tasks(update.message.reply_text)
```

— Отправка счёта —

```
async def send_scores(reply_func):
    data = load_data()
    scores = data["scores"]
    loser_id = get_loser(scores)

    lines = ["🏆 *Счёт за месяц:*\n"]
    sorted_people = sorted(PEOPLE.keys(), key=lambda p: scores[p], reverse=True)

    for i, pid in enumerate(sorted_people):
        p = PEOPLE[pid]
        s = scores[pid]
        medal = ["🥇", "🥈", "🥉"][i] if i < 3 else " "
        cinema = "🎬" if pid == loser_id and s == scores[loser_id] else ""
        lines.append(f"{medal} {p['emoji']} *{p['name']}* — {s} бал.{cinema}")

    lines.append(f"\n⚠️ У кого меньше всех баллов — ведёт в кино!\n🎬 Сейчас это: ")

    await reply_func("\n".join(lines), parse_mode="Markdown")
```

— Список задач —

```
async def send_tasks(reply_func):
    keyboard = []
    for tid, task in TASKS.items():
        p = PEOPLE[task["person"]]
```

```

        btn_text = f"{task['emoji']} {p['name']}: {task['name']} (+{task['pts']})"
        keyboard.append([InlineKeyboardButton(btn_text, callback_data=f"done_{tid}")

keyboard.append([InlineKeyboardButton("🏆 Счёт", callback_data="show_scores")

await reply_func(
    "📝 *Выбери выполненную задачу:*,
    parse_mode="Markdown",
    reply_markup=InlineKeyboardMarkup(keyboard)
)

# ——— Обработка нажатий —————

async def button_handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    data_cb = query.data

    if data_cb == "show_scores":
        await send_scores(query.message.reply_text)

    elif data_cb == "show_tasks":
        await send_tasks(query.message.reply_text)

    elif data_cb == "show_history":
        data = load_data()
        history = data.get("history", [])
        if not history:
            await query.message.reply_text("📖 История пока пуста.")
            return
        lines = ["📖 *Последние 10 действий:*\n"]
        for entry in reversed(history[-10:]):
            lines.append(f"• {entry}")
        await query.message.reply_text("\n".join(lines), parse_mode="Markdown")

    elif data_cb.startswith("done_"):
        tid = data_cb.replace("done_", "")
        task = TASKS.get(tid)
        if not task:
            await query.message.reply_text("Задача не найдена.")
            return

        data = load_data()
        today = datetime.now().strftime("%Y-%m-%d")

        # Проверка лимита в день (для игрушек — max 2)
        max_per_day = task.get("max_per_day", 1)

```

```

count_key = f"{tid}_{today}"
today_count = data.get("today_count", {})
done_today = today_count.get(count_key, 0)

if done_today >= max_per_day:
    limit_msg = "2 паза" if max_per_day == 2 else "1 паз"
    await query.message.reply_text(
        f"🚫 Эта задача уже выполнена сегодня ({limit_msg} максимум)."
    )
    return

# Начисляем баллы
pid = task["person"]
data["scores"][pid] += task["pts"]
today_count[count_key] = done_today + 1
data["today_count"] = today_count

# Запись в историю
p = PEOPLE[pid]
now_str = datetime.now().strftime("%d.%m %H:%M")
entry = f"{task['emoji']} {p['name']} - {task['name']} *+{task['pts']} ба
data["history"].append(entry)
save_data(data)

# Ответ в чат
scores = data["scores"]
loser_id = get_loser(scores)
loser_name = PEOPLE[loser_id]["name"]

msg = (
    f"✅ *Выполнено!*\n\n"
    f"{task['emoji']} {p['emoji']} *{p['name']}* - {task['name']}\n"
    f"$ +{task['pts']} бал → итого: *{scores[pid]} бал.*\n\n"
    f"🏆 Счёт:\n"
)

for ppid, pp in PEOPLE.items():
    cinema = " 🎬" if ppid == loser_id else ""
    msg += f" {pp['emoji']} {pp['name']}: {scores[ppid]} бал.{cinema}\n"

msg += f"\n🎬 В кино ведёт: *{loser_name}*"

keyboard = [
    InlineKeyboardButton("📁 Ещё задачи", callback_data="show_tasks"),
    InlineKeyboardButton("🏆 Счёт", callback_data="show_scores")]
]

await query.message.reply_text(msg, parse_mode="Markdown",
                                reply_markup=InlineKeyboardMarkup(keyboard))

```

— Запуск —

```
def main():
    token = os.environ.get("BOT_TOKEN")
    if not token:
        raise ValueError("Нет BOT_TOKEN в переменных окружения!")

    app = Application.builder().token(token).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("score", scores_cmd))
    app.add_handler(CommandHandler("tasks", tasks_cmd))
    app.add_handler(CallbackQueryHandler(button_handler))

    logger.info("Бот запущен!")
    app.run_polling()

if __name__ == "__main__":
    main()
```